

1. ¿Qué busca un ataque de "Whaling"?

- a. Comprometer a objetivos de alto perfil o ejecutivos (C-Level) mediante ingeniería social muy sofisticada.
- b. Atacar servidores con grandes bases de datos.
- c. Enviar spam a listas de correo grandes.
- d. Pescar ballenas ilegalmente.

2. Una aplicación utiliza una clave de cifrado hardcodeda (fija en el código) para cifrar datos sensibles: "CONSTANTE CLAVE CIFRADO = "MySuperSecretKey". ¿Cuál es el principal problema de seguridad?

- a. El algoritmo es lento.
- b. la clave es muy larga
- c. Uso de clave hardcodeda (Hardcoded Key): vulnerabilidad crítica si se decompila la app.
- d. No hay problema.

3. ¿Cuál es el propósito de realizar "Threat Hunting" (Caza de Amenazas) proactivamente en una organización?

- a. Buscar activamente indicadores de compromiso (IoCs) o actividades maliciosas dentro de la red y los sistemas que *no* han sido detectados por las herramientas de seguridad automatizadas existentes.
- b. Reemplazar la necesidad de tener un SIEM (Security Information and Event Managemen.
- c. c .Desarrollar parches de seguridad para vulnerabilidades zero-day.
- d. Realizar pruebas de penetración contra la propia organización.

4. Explique por qué depender únicamente de la validación de entradas en el lado del diente (frontend) es insuficiente y qué principio de seguridad viola fundamentalmente esta práctica.

- a. Porque los navegadores antiguos no la soportan.
- b. Porque puede ser eludida (bypassed) fácilmente interceptando la petición: la seguridad real debe estar en el backend.
- c. Porque consume muchos datos móviles.
- d. Porque es lenta

5. **¿Qué buena práctica se debe seguir al manejar errores en una aplicación segura?**
- a. Ignorar los errores menores
 - b. Almacenar los errores en la base de datos del cliente
 - c. Mostrar el error completo al usuario
 - d. Registrar los errores detallados en logs seguros
6. **¿Qué técnica de Ingeniería Social implica el uso de llamadas telefónicas para engañar a la víctima y obtener información confidencial?**
- a. Phishing.
 - b. Vishing (Voice Phishing).
 - c. Smishing
 - d. Dumpster Diving.
7. **¿Cuál es el objetivo principal de una “Matriz de Trazabilidad de Requisitos de seguridad”?**
- a. Identificar a los desarrolladores que escribieron el código.
 - b. Rastrear la ubicación física de los servidores.
 - c. Asegurar que cada requisito de seguridad esté vinculado a casos de uso, código y pruebas que lo verifiquen.
 - d. Medir el tiempo de respuesta de la base de datos.
8. **¿Cuál es el objetivo principal del ciclo de vida del desarrollo seguro?**
- a. Reducir los costos de infraestructura
 - b. Aumentar la velocidad de desarrollo
 - c. Cumplir únicamente con auditorías
 - d. Integrar la seguridad en todas las fases del ciclo de vida del software
9. **¿Qué requisito de seguridad busca asegurar que una entidad no pueda negar la autoría de una acción realizada (ej. firma de contrato, transferencia)?**
- a. No repudio.
 - b. Autorización.
 - c. Confidencialidad.
 - d. Disponibilidad.

10. Durante la fase de requisitos de un S-SDLC. ¿Cuál es la actividad de seguridad más relevante para prevenir fallos de diseño costosos posteriormente?

- a. Configurar firewalls de red perimetrales.
- b. Escribir pruebas unitarias para cada función.
- c. Implementar pruebas de penetración automatizadas.
- d. Realizar un modelado de amenazas inicial basado en los casos de uso y requerimientos funcionales.

11. ¿Qué patrón de diseño de seguridad ayuda a reducir la superficie de ataque canalizando todas las solicitudes a través de un único punto de entrada y validación?

- a. Modelo Vista Controlador (MVO).
- b. Singleton
- c. Active Record
- d. intercepting Filter / API Gateway Centralizado.

12. ¿Qué tipo de malware se disfraza como software legítimo pero realiza acciones maliciosas una vez ejecutado?

- a. Troyano (Trojan Horse).
- b. Adware.
- c. spyware.
- d. Ransomware.

13. ¿Qué tipo de ataque intenta agotar los recursos de un servidor (CPU, memoria, conexiones) enviando un gran volumen de tráfico malicioso desde una “única* fuente?

- a. Man-in-the-Middie (MitM).
- b. Inyección SQL
- c. Denegación de Servicio (DoS).
- d. Denegación de Servicio Distribuido (DDOS).

14. ¿Qué describe la “Economía de Mecanismos” (Economy of Mechanism) como principio de diseño seguro?

- a. Basar la seguridad en algoritmos públicamente conocidos y revisados (Diseño Abierto).
- b. Asegurar que todos los accesos sean mediados por un control centralizado (Mediación Completa).
- c. Mantener el diseño de seguridad lo más simple y pequeño posible. Diseños complejos son más difíciles de probar y más propensos a contener errores.
- d. Utilizar múltiples mecanismos de seguridad redundantes (Defensa en Profundidad).

15. ¿Qué técnica utiliza un atacante en un ataque “Credential Stuffing”?

- a. Explotar una vulnerabilidad de inyección SQL para extraer hashes de contraseñas.
- b. Interceptar contraseñas enviadas sobre HTTP.
- c. Adivinar contraseñas usando listas de palabras comunes (ataque de diccionario).
- d. Probar credenciales (usuario/contraseña) robadas de una brecha de datos en “otros” sitios web, aprovechando la reutilización de contraseñas por parte de los usuarios.

16. ¿Qué ataque específico podría prevenirse usando la cabecera HTTP X-Content-Type-Options: nosniff?

- a. Fuga de información a través de la cabecera "Server"
- b. Cross-site scripting (XSS) reflejado.
- c. Ataques de "MIME Sniffing", donde el navegador intenta adivinar el tipo de contenido de un recurso ignorando la cabecera "Content-Type,, lo que podría llevar a interpretar un archivo de texto como HTML o script si contiene ciertos patrones.
- d. Clickjacking

17. ¿Por qué no se debe depender únicamente de la validación del lado del cliente?

- a. Porque puede ser fácilmente manipulada o deshabilitada
- b. Porque hace más lento el procesamiento
- c. Porque genera errores de compatibilidad
- d. Porque aumenta el consumo de recursos

18. El siguiente pseudo-código construye una consulta de base de datos usando concatenación de strings: id_usuario = LEER_PARAMETRO("user-id") "query = "SELECT * FROM users WHERE id = " + id_usuario + "" "EJECUTAR SQL(query)". ¿Cuál es la vulnerabilidad y la solución obligatoria?

- a. Cross-Site Scripting (XSS); Solución: Usar codificación de salida HTML.
- b. Inyección de Comandos; Solución: Evitar llamadas al sistema operativo.
- c. Path Traversal; Solución: Validar que no contenga ./.
- d. Inyección SQL (SQL Injection); Solución: Usar Sentencias Preparadas (Prepared Statements) o consultas parametrizadas**

19. ¿Qué tipo de prueba de seguridad se enfoca en verificar que los controles de acceso funcionen como se espera, asegurando que los usuarios solo puedan acceder a los recursos y realizar las acciones para las que están autorizados?

- a. Pruebas de Inyección.
- b. Pruebas de Control de Acceso / Autorización.**
- c. Pruebas de Gestión de sesión.
- d. Pruebas de Criptografía.

20. Compare y contraste SAST y SCA en términos de lo que analizan y el tipo de vulnerabilidades que típicamente encuentran.

- a. SAST analiza la red; SCA analiza la base de datos.
- b. SAST analiza el código fuente propio; SCA analiza las librerías de terceros (dependencias)**
- c. SAST es dinámico; SCA es estático.
- d. Ambas hacen lo mismo.

21. ¿Qué técnica criptográfica produce una salida de longitud fija (hash) a partir de una entrada de longitud variable, siendo computacionalmente inviable encontrar la entrada original o dos entradas que produzcan el mismo hash?

- a. Función Hash Criptográfica (ej. SHA-256).**
- b. Cifrado Asimétrico (ej. RSA).
- c. Cifrado simétrico (ej AES).
- d. Esteganografía.

22. En el contexto de pruebas de seguridad, ¿Qué significa “Fuzzing”:

- a. Un análisis de las dependencias de terceros.
- b. Un escaneo dinámico que sigue enlaces y envía payloads conocidos.
- c. Una revisión manual detallada del código fuente.
- d. Una técnica de prueba automatizada que consiste en proporcionar datos inválidos, esperados o aleatorios ("fuzz") como entrada a un programa para monitorear caídas, aseveraciones fallidas o posibles vulnerabilidades de seguridad.

23. En una transacción compleja (ej. comprar producto y ganar puntos), ¿cómo puede un mal uso de 'SAVEPOINT' y 'ROLLBACK TO SAVEPOINT' causar pérdida de integridad?

- a. Haciendo 'ROLLBACK' de toda la transacción si los puntos no están disponibles.
- b. Los 'SAVEPOINT' bloquean la base de datos y causan Dos.
- c. Permitiendo que la parte de “ganar puntos” se confirme (COMMIT), pero la parte de ‘pagar producto’ se revierta a un 'SAVEPOINT' previo, generando puntos sin compra.
- d. Los 'SAVEPOINT' solo se usan para acelerar el 'COMMIT' final.

24. ¿Qué ventaja principal ofrece el uso de “Prepared Statements” (o consultas parametrizadas) en la interacción con bases de datos?

- a. Permite ejecutar múltiples sentencias SQL en una sola llamada.
- b. Elimina la necesidad de validar los tipos de datos en la aplicación.
- c. Mejora significativamente el rendimiento de todas las consultas a la base de datos.
- d. Previene eficazmente las vulnerabilidades de Inyección SQL al separar el código SQL de los datos del usuario.

25. ¿Cuál es el propósito de la herramienta OWASP ZAP (Zed Attack Proxy)?

- a. Verificar la seguridad de las dependencias de terceros (SCA).
- b. Analizar el código fuente de forma estática (SAST).

- c. Actuar como un proxy de interceptación y realizar escaneos de seguridad dinámicos (DAST) en aplicaciones web para encontrar vulnerabilidades.
- d. Modelar amenazas en la fase de diseño.

26. Una aplicación permite ingresar el precio de un producto. Si un usuario ingresa accidentalmente "100.00a" en lugar de "100.00", ¿qué control de seguridad previene esta modificación "sin dolo" que corrompe la integridad del dato?

- a. Cifrado de la base de datos.
- b. Autenticación Multifactor (MFA).
- c. Uso de HITPS.
- d. Validación estricta del tipo de dato y formato (Whitelist) en el backend.

27. ¿Qué requisito de auditoría (logging) es crítico para el análisis forense?

- a. registrar únicamente los errores del sistema.
- b. No registrar nada para ahorrar espacio.
- c. Registrar eventos de éxito y fallo en autenticación y autorización, incluyendo "Quién", "Qué", "Cuándo" y "Desde dónde".
- d. Registrar contraseñas en texto plano.

28. Un servidor está configurado con "add header X-Frame-Options "SAMEORIGIN";. ¿Qué previene esta cabecera?

- a. Clickjacking (al impedir que la página se cargue en iframes externos).
- b. XSS.
- c. SQL Injection.
- d. DoS.

29. Una aplicación permite comentarios y los muestra usando una función que interpreta HTML (FUAR HTML EN PANTALLA",. Si "comentario_usuario" contiene ", ¿qué sucederá?

- a. Se muestra el texto del script pero no se ejecuta.

- b. Error de base de datos.
- c. Naad, el navegador lo bloquea.
- d. Se ejecuta el script (XSS Reflejado/Almacenado).

30. ¿Qué tipo de vulnerabilidad se explota en un ataque “HTTP Request smuggling”?

- a. El secuestro de una sesión HTTP válida utilizando un token robado.
- b. La inconsistencia en la interpretación de límites de solicitudes HTTP (ej, “Content-Length” vs “Transfer-Encoding) entre proxies y servidores backend, permitiendo a un atacante enviar una segunda solicitud maliciosa dentro de una primera solicitud aparentemente inocua.
- c. La inyección de cabeceras HTTP adicionales para realizar ataques como cache poisoning o XSS.
- d. La falsificación de una solicitud desde el navegador de la víctima a otro sitio donde está autenticada (CSRF).

31. En el desarrollo web seguro, ¿qué papel juega el protocolo TLS/SSL y qué medida de seguridad fundamental garantiza?

- a. TLS/SSL solo garantiza que el servidor verifique la identidad del cliente (navegador), evitando que bots maliciosos accedan al sitio web.
- b. TLS/SSL utiliza únicamente criptografía simétrica para el handshake inicial, lo que garantiza la no repudiación de las transacciones web.
- c. El principal papel de TLS/SSL es cifrar los datos una sola vez antes de que salgan del cliente; una vez en el servidor, el protocolo ya no tiene ninguna función de seguridad.
- d. Es fundamental para proteger la comunicación entre el cliente (navegador) y el servidor. Garantiza la confidencialidad e integridad de los datos en tránsito.

**32. Analice el siguiente pseudo-código: “url usuario = LEER_P
ARAMETRO_URL('imageUrl'; datos imagen =
HACER_REQUEST_HTTP_GET (url_usuario) . Si un atacante proporciona
“url usuario = "http://169.254.169.254/latest/meta-data/" o
“http://localhost/admin” , ¿qué vulnerabilidad se está explotando?**

- a. Client-side Request Forgery (CSRF)

- b. Server-Side Request Forgery (SSRP)
- c. Cross-site Scripting (455)
- d. Open Redirect (Redirección Abierta)

33. Una aplicación web almacena un token de sesión sensible en el "Almacenamiento Local" del navegador (accesible por scripts). ¿Por qué esto se considera menos seguro que usar cookies con el atributo "HttpOnly?"

- a. Porque "localStorage" se borra al cerrar el navegador.
- b. Porque "localStorage" es accesible vía Javascript haciéndolo vulnerable a robo por XSS.
- c. Porque "localStorage" tiene menos capacidad.
- d. Porque las cookies son más rápidas.

34. En seguridad móvil, ¿qué es el "Swizzling" de métodos (Objective-C/Swift) y por qué es un riesgo?

- a. La optimización automática de código realizada por el compilador.
- b. Es una técnica que permite reemplazar la implementación de un método existente en tiempo de ejecución. Es un riesgo porque un atacante (o malware) podría usarlo para interceptar llamadas sensibles, modificar lógica de seguridad o robar datos dentro de la app.
- c. Un tipo de ofuscación de código que renombra variables.
- d. Una forma de gestionar la memoria en iOS.

35. ¿Cuál es el riesgo de seguridad si una aplicación móvil almacena claves criptográficas directamente en el código fuente o en archivos de configuración no protegidos?

- a. Las claves podrían entrar en conflicto con otras variables del sistema.
- b. El tamaño final del paquete de la aplicación aumentará considerablemente.
- c. Las claves pueden ser fácilmente extraídas mediante ingeniería inversa del binario de la aplicación, comprometiendo todos los datos cifrados con ellas.
- d. El rendimiento de la aplicación disminuirá debido al acceso constante a las claves.

36. Una aplicación web utiliza OAuth 2.0 (Authorization Code Grant) para permitir que los usuarios inicien sesión con Google. Explique brevemente el flujo y mencione dos puntos críticos donde pueden ocurrir vulnerabilidades si no se implementa correctamente.

- a. sql Injection.
- b. XSS.
- c. Man-in-the-Middle.
- d. Cross Site Request Forgery (CSRF) en el paso de callback.

37. En el contexto de OWASP API Security Top 10, ¿qué implica la “Gestión Inadecuada del Inventario” (API2023)?

- a. La falta de limitación de velocidad (Rate Limiting) en las APIs
- b. El almacenamiento inseguro de las claves API utilizadas por los clientes.
- c. La falta de visibilidad y documentación sobre todas las APIs expuestas por la organización (incluyendo versiones antiguas, endpoints olvidados, APIs “shadow”), lo que lleva a que no se apliquen controles de seguridad o parches adecuados.
- d. El uso de un número excesivo de endpoints en una sola API.

38. Según el OWASP API security Top 10, ¿qué vulnerabilidad ocurre cuando una API no valida correctamente si el usuario autenticado tiene permiso para acceder a un “objeto específico” (ej. registro de base de datos)?

- a. API2023 - Broken Function Level Authorization.
- b. API2023 - Broken Authentication.
- c. API2023 - Broken Object Level Authorization (BOLA).
- d. API2023 - Broken Object Property Level Authorization.

39. ¿Cuál es el beneficio de seguridad de usar contenedores Docker con el flag —read-only para el sistema de archivos raíz?

- a. Permite al contenedor escribir logs en cualquier ubicación.
- b. Facilita la actualización de paquetes dentro del contenedor en ejecución.
- c. Mejora el rendimiento de lectura/escritura del contenedor
- d. Previene modificaciones en el sistema de archivos del contenedor en tiempo de ejecución, dificultando la persistencia de malware o la alteración de archivos críticos.

40. En el contexto de la seguridad en la nube, ¿qué describe el "Modelo de Responsabilidad Compartida" (shared Responsibility Model)?

- a. Define qué aspectos de la seguridad son responsabilidad del proveedor de la nube (ej. seguridad "de la nube: infraestructura física, hipervisor) y cuáles son responsabilidad del cliente (ej. seguridad "en* la nube: datos, configuración de SO, red, IAM,
- b. Un modelo donde toda la seguridad recae únicamente en el cliente.
- c. Un modelo donde toda la seguridad es gestionada automáticamente por el proveedor de la nube.
- d. Un acuerdo legal que exime al proveedor de nube de cualquier responsabilidad por brechas de seguridad.

41. En el contexto de la seguridad de 1A ¿qué implica la "Robustez" de un modelo?

- a. La capacidad del modelo para mantener su rendimiento (precisión, etc) incluso cuando se enfrenta a entradas inesperadas, ruidosas o adversariales (manipuladas).
- b. El tamaño del modelo en términos de parámetros o espacio en disco.
- c. la velocidad con la que el modelo puede procesar nuevas entradas (latencia).
- d. la facilidad con la que se puede entender cómo el modelo toma sus decisiones (interpretabilidad)

42. En el contexto de la seguridad de 1A ¿qué es el "Aprendizaje Federado" (Federated Learning)?

- a. Un método para entrenar modelos usando exclusivamente datos sintéticos generados centralmente.
- b. Un sistema para auditar modelos de 1A desplegados en producción usando datos federales.
- c. Una arquitectura donde múltiples modelos de 1A compiten entre sí para mejorar la precisión.
- d. Una técnica de entrenamiento de modelos de Machine Learning donde los datos permanecen en los dispositivos locales (ej. teléfonos) y solo las actualizaciones del modelo (gradientes, pesos) se envían a un servidor central para agregación, mejorando la privacidad.

43. ¿Cuál es el propósito de definir "Historias de Usuario de Seguridad" (Security User Stories) o "Evil User Stories" en metodologías ágiles?

- a. Visibilizar los requisitos de seguridad en el Backlog del producto para que sean priorizados y desarrollados.
- b. Reemplazar el modelado de amenazas.
- c. Hacer el trabajo más difícil para los desarrolladores.
- d. Documentar los bugs encontrados en producción.

44. ¿Qué patrón de diseño de seguridad ayuda a reducir la superficie de ataque canalizando todas las solicitudes a través de un único punto de entrada y validación?

- a. Modelo Vista Controlador (MVC).
- b. Singleton.
- c. Active Record.
- d. Intercepting Filter / API Gateway Centralizado.

45. ¿Cuál es el propósito de un "Honeypot" en seguridad de redes?

- a. Un servidor centralizado para recolectar logs de seguridad.
- b. Un sistema para realizar copias de seguridad automáticas de datos críticos.
- c. Actuar como un señuelo, un sistema intencionalmente vulnerable diseñado para atraer y atrapar a atacantes, permitiendo estudiar sus métodos y distraerlos de los sistemas reales.
- d. Un tipo de firewall avanzado que utiliza inteligencia artificial.

46. ¿Qué define la etapa de "Diseño" en el SDLC seguro?

- a. La implementación de la aplicación en el entorno de producción.
- b. La creación de la arquitectura de la aplicación, incluyendo modelos de datos, interfaces y la planificación de controles de seguridad específicos.
- c. La escritura del código fuente de la aplicación.
- d. La evaluación de riesgos y definición de requisitos iniciales.

47. Una aplicación genera IDs de sesión usando la hora actual del sistema (ej. "OBTENER_HORA_ACTUAL_MILISEGUNDOS()"). ¿Por qué es inseguro?

- a. No, porque es muy corto.
- b. Sí, la hora cambia siempre.
- c. No, es predecible para un atacante (Session Guessing/Hijacking).
- d. Sí, si tiene milisegundos.

48. ¿Cuál de las siguientes es una métrica clave en la planificación de la continuidad del negocio y recuperación ante desastres (BCP/DRP) que define la cantidad máxima de pérdida de datos tolerable (medida en tiempo)?

- a. Recovery Time Objective (RTO).
- b. Service Level Agreement (SLA).
- c. Recovery Point Objective (RPO).
- d. Maximum Tolerable Downtime (MTD).

49. ¿Cuál es un error de implementación criptográfica común que puede llevar a vulnerabilidades?

- a. El error más común es usar algoritmos que son demasiado modernos o complejos; los algoritmos antiguos como DES son en realidad más seguros porque han sido probados por más tiempo.
- b. Reutilizar el mismo vector de inicialización (IV) o nonce en el cifrado de bloques con la misma clave es una forma eficiente de acelerar el proceso de cifrado sin comprometer la seguridad.
- c. Almacenar claves criptográficas directamente en el código fuente de la aplicación es una práctica segura porque solo los desarrolladores tienen acceso al repositorio.
- d. Un error común es el uso de claves débiles o predecibles, o la mala gestión de las claves (como almacenarlas en texto plano)

50. En el modelado de amenazas con DREAD, ¿qué evalúa la 'A' (Affected Users)?

- a. La cantidad o porcentaje de usuarios que se verían afectados si la amenaza se materializa..
- b. La facilidad para reproducir el ataque.

- c. la facilidad con la que un atacante puede descubrir la vulnerabilidad.
- d. El daño potencial total causado por la amenaza (financiero, reputacional, etc).

51. ¿Qué implica el principio de "Economía de Mecanismos" en el diseño de software seguro?

- a. Mantener el diseño de seguridad lo más simple y pequeño posible, ya que la complejidad aumenta la probabilidad de errores y vulnerabilidades.
- b. No gastar dinero en herramientas de seguridad.
- c. Usar el software más barato disponible.
- d. Usar algoritmos de cifrado cortos para ahorrar CPU.

52. ¿Qué tecnología de seguridad de red permite crear una conexión cifrada y segura entre un dispositivo remoto (ej. empleado trabajando desde casa) y la red corporativa?

- a. Red Privada Virtual (VPN - Virtual Private Network).
- b. Balanceador de Carga (Load Balancer).
- c. Firewall de Aplicación Web (WAF - Web Application Firewall).
- d. Sistema de Detección de Intrusos (IDS - Intrusion Detection System).

53. ¿Qué representa un "Activo" en el contexto de la gestión de riesgos de seguridad?

- a. Una amenaza potencial como un hacker o malware.
- b. Un control de seguridad implementado, como un firewall.
- c. Una vulnerabilidad conocida en un sistema operativo.
- d. Cualquier recurso de valor para la organización que necesita ser protegido (ej. Datos de clientes, propiedad intelectual, reputación, infraestructura, sistemas).

54. ¿Qué buena práctica se debe seguir al manejar errores en una aplicación segura?

- a. Almacenar los errores en la base de datos del cliente
- b. Registrar los errores detallados en logs seguros**
- c. Ignorar los errores menores
- d. Mostrar el error completo al usuario

55. ¿Cuál es el objetivo principal del ciclo de vida del desarrollo seguro?

- a. Aumentar la velocidad de desarrollo
- b. Integrar la seguridad en todas las fases del ciclo de vida del software**
- c. Reducir los costos de infraestructura
- d. Cumplir únicamente con auditorías

56. ¿Qué práctica recomendada por OWASP ayuda a prevenir que un atacante utilice múltiples intentos para adivinar códigos de verificación enviados por SMS o email (ej. para MFA o recuperación de contraseña)?

- a. No registrar los intentos fallidos para no alertar al atacante.
- b. Permitir que los códigos de verificación sean válidos durante varias horas.
- c. Enviar códigos de verificación muy cortos (4 dígitos).
- d. Implementar Rate Limiting (limitación de intentos) y CAPTCHA después de pocos intentos fallidos.**

57. ¿Qué técnica criptográfica es fundamental para asegurar la integridad de un activo de información (como un documento o un log), permitiendo detectar cualquier modificación no autorizada (con dolo o sin dolo)?

- a. Almacenamiento en la nube (ej. S3).
- b. Compresión de datos (ej. ZIP).
- c. Función Hash Criptográfica (ej. SHA-256).**
- d. Cifrado Simétrico (ej. AES).

58. Una aplicación almacena contraseñas usando el siguiente pseudo-código: "hash_almacenado = FUNCION HASH MDs/(passwordusuario)". ¿Cuál es el principal problema de seguridad?

- a. MDS es criptográficamente roto y rápido (fácil de crackear con rainbow tables).
- b. MD5 es demasiado lento.
- c. No es compatible con UTF-8.
- d. Ocupa mucho espacio.

59. ¿Qué ataque específico podría prevenirse usando la cabecera HTTP 'X-Content-Type-Options:nosniff'?

- a. Cross-Site Scripting (XSS) reflejado.
- b. Fuga de información a través de la cabecera "Server"
- c. Ataques de "MIME sniffing", donde el navegador intenta adivinar el tipo de contenido de un recurso ignorando la cabecera "Content-Type", lo que podría llevar a interpretar un archivo de texto como HTML o script si contiene ciertos patrones.
- d. Clickjacking.

60. Analice el siguiente pseudo-código de un endpoint de API que borra usuarios: 'FUNCION ELIMINAR_USUARIO(id_a_borrar, usuario logueado): * // No se verifica si 'usuario logueado' tiene permiso para borrar 'id.a borrar' * BORRAR_DE_BD(id_a_borrar)'. ¿Qué táctica de ataque se está facilitando?

- a. Phishing.
- b. Man-in-the-Middle en la conexión.
- c. Escalada de Privilegios / Modificación de datos no autorizada (un usuario normal borrando a otro).
- d. Denegación de Servicio (DoS) por consumo de CPU.

61. ¿Qué ventaja principal ofrece el uso de "Prepared Statements" (o consultas parametrizadas) en la interacción con bases de datos?

- a. Elimina la necesidad de validar los tipos de datos en la aplicación.
- b. Previene eficazmente las vulnerabilidades de Inyección SQL al separar el código SQL de los datos del usuario.
- c. Permite ejecutar múltiples sentencias SQL en una sola llamada.
- d. Mejora significativamente el rendimiento de todas las consultas a la base de datos.

62. Una aplicación genera un token de reseteo de contraseña y lo envía en la URL: "http://example.com/reset?token=XYZ123". ¿Cuál es el riesgo principal de usar "http" en lugar de "https"?

- a. El token es muy corto.
- b. Intercepción del token (Sniffing) al viajar en texto plano por la red.
- c. El navegador bloqueará la página.
- d. El usuario no sabrá usarlo.

63. Un desarrollador quiere utilizar una librería de terceros para procesar datos, pero esta no ha sido actualizada en más de dos años. ¿Cuál es la acción más recomendable desde la perspectiva de desarrollo seguro?

- a. Clonar el repositorio y modificarlo manualmente para evitar dependencias externas.
- b. Analizar vulnerabilidades conocidas mediante herramientas como OWASP Dependency-Check o Snyk antes de incluirla.
- c. Usar la librería porque sigue funcionando correctamente.
- d. Revisar la reputación del autor y continuar si es confiable

64. ¿Qué tipo de vulnerabilidad explota un atacante que logra incluir un archivo local del servidor (ej. /etc/passwd') en la respuesta de una página web debido a una validación insuficiente del nombre de archivo?

- a. Remote File Inclusion (RFI).
- b. Path Traversal.
- c. Server-Side Request Forgery (SSRF).
- d. Local File Inclusion (LFI).

65. Un servidor está configurado con 'add_header X-Frame-Options "SAMEORIGIN";. ¿Qué previene esta cabecera?

- a. Clickjacking (al impedir que la página se cargue en iframes externos).
- b. SQL Injection.
- c. Dos.
- d. XSSs.

66. ¿Cuál es el propósito de la cabecera HTTP "Referrer-Policy"?

- a. Controlar cuánta información de la URL de origen (referrer) se incluye en las solicitudes que se hacen desde la página actual a

otros sitios. Ayuda a prevenir la fuga de información sensible en las URLs.

- b. Prevenir el Clickjacking (como X-Frame-Options).
- c. Mitigar XSS reflejado (como X-XSS-Protection, aunque obsoleto).
- d. Forzar el uso de HTTPS (como HSTS).

67. ¿Cuál es el propósito de la cabecera HTTP “Content-Security-Policy” (CSP)?

- a. Forzar el uso de HTTPS en todas las comunicaciones futuras (similar a HSTS).
- b. Evitar que el navegador interprete archivos con un tipo MIME incorrecto (como XContent-Type-Options).
- c. Controlar qué información de referente se envía en las solicitudes.
- d. Definir políticas que controlan qué recursos (scripts, CSS, imágenes) puede cargar el navegador, mitigando XSS y otros ataques de inyección.

68. Analice el siguiente pseudo-código. Si la variable “InputUsuario” contiene un intento de Inyección SQL (ej. * 0 1=1 --), ¿qué sucederá?
“InputUsuario = LEER PARAMETRO(“userNu”) - “query = “SELECT name FROM users WHERE userNu = ?” “EJECUTAR SQL(query,PARAMETRO(1, InputUsuario))”

- a. La base de datos arrojará un error de sintaxis SQL.
- b. El sistema se bloqueará por seguridad.
- c. La entrada será tratada estrictamente como datos literales (texto), neutralizando el intento de inyección.
- d. Se ejecutará la inyección SQL y devolverá todos los usuarios.

69. ¿Cuál de las siguientes prácticas debe aplicarse al diseñar un flujo de autenticación seguro?

- a. Implementar multifactor (MFA) y expiración de sesión
- b. Almacenar contraseñas en cookies
- c. Usar tokens permanentes
- d. Permitir autenticación solo por IP

70. En el contexto de pruebas de seguridad, ¿qué significa “Fuzzing”?

- a. Una revisión manual detallada del código fuente.
- b. Un escaneo dinámico que sigue enlaces y envía payloads conocidos.
- c. Una técnica de prueba automatizada que consiste en proporcionar datos inválidos, inesperados o aleatorios (*fuzz') como entrada a un programa para monitorear caídas, aseveraciones fallidas o posibles vulnerabilidades de seguridad.
- d. Un análisis de las dependencias de terceros.

71. Una aplicación ejecuta comandos del sistema: “directorio = LEER PARAMETRO_URL("dir")” comando_os = "LISTAR ARCHIVOS * + directorio” 'EJECUTAR COMANDO. Os(comando.os)'. ¿Qué vulnerabilidad introduce?

- a. Inyección SQL
- b. Path Traversal.
- c. Inyección de Comandos (OS Command Injection).
- d. Crosssite Scripting (XSS).

72. Una aplicación web permite a los usuarios subir imágenes de perfil. Explique dos vulnerabilidades de seguridad que podrían surgir si el proceso de subida no se implementa de forma segura y cómo mitigarlas.

- a. Cambiando el nombre del archivo y validando el tipo MIME real y contenido (Magic Bytes).
- b. Guardando el archivo en la misma carpeta del código.
- c. Comprimiendo el archivo.
- d. Validando solo la extensión del archivo.

73. ¿Qué describe mejor un ataque de "Clickjacking"?

- a. Engañar a un usuario para que haga clic en algo diferente a lo que percibe visualmente, superponiendo elementos invisibles (ej. iframes) sobre la interfaz legítima, para realizar acciones no deseadas en otro sitio donde el usuario está autenticado.
- b. Inyectar scripts maliciosos en una página web (XSS).

- c. Forzar al navegador del usuario a realizar solicitudes no deseadas (CSRF).
- d. Secuestrar la sesión del usuario robando sus cookies.

74. ¿Cuál de los siguientes mecanismos protege mejor la integridad de los datos transmitidos entre cliente y servidor?

- a. Token JWT sin firma
- b. HTTPS/TLS**
- c. Compresión de datos
- d. Hashing MD5

75. Una aplicación permite comentarios y los muestra usando una función que interpreta HTML ("CFIJAR_HTML_EN_PANTALLA"). Si "comentario_usuario" contiene "", ¿qué sucederá?

- a. Se muestra el texto del script pero no se ejecuta.
- b. Se ejecuta el script (XSS Reflejado/Almacenado).**
- c. Nada, el navegador lo bloquea.
- d. Error de base de datos.

76. ¿Qué categoría del OWASP Top 10 cubre los riesgos de fallos de integridad en la cadena responder de suministro de software? (ej. la falta de verificación de firmas digitales en las dependencias, o la modificación de artefactos en el pipeline de CI/CD sin ser detectada).

- a. A05:2021 - Security Misconfiguration (Configuración Incorrecta de Seguridad).
- b. A06:2021 - Vulnerable and Outdated Components (Componentes Vulnerables y Obsoletos).
- c. A08:2021 - Software and Data Integrity Failures (Fallos de Integridad de Software y Datos).**
- d. A02:2021 - Cryptographic Failures (Fallos Criptográficos).

77. En seguridad móvil, ¿qué es el "Swizzling* de métodos (Objective-C/Swift) y por qué es un riesgo?

- a. Es una técnica que permite reemplazar la implementación de un método existente en tiempo de ejecución. Es un riesgo porque un atacante (0 malware) podría usarlo para interceptar llamadas sensibles, modificar lógica de seguridad o robar datos dentro de la app.
- b. Una forma de gestionar la memoria en iOS.
- c. La optimización automática de código realizada por el compilador.
- d. Un tipo de ofuscación de código que renombra variables.

78. ¿Qué información sensible podría filtrarse si una aplicación móvil no deshabilita correctamente las funciones de depuración (ej. logging verbose, modo debug) en la versión de producción?

- a. Información sobre la versión del sistema operativo del dispositivo.
- b. El identificador único del dispositivo (IMEI/UDID).
- c. Claves API, tokens, datos internos, endpoints privados, lógica de negocio detallada, que pueden ser útiles para un atacante.
- d. Únicamente mensajes de error genéricos del sistema operativo.

79. ¿Cuál es el riesgo principal de no implementar “Rate Limiting” en una API de autenticación o de recuperación de contraseña?

- a. Aumenta la latencia promedio de respuesta de la API.
- b. Requiere almacenar hashes de contraseñas más complejos.
- c. Facilita ataques de Fuerza Bruta y Credential Stuffing al permitir un número ilimitado de intentos en poco tiempo.
- d. Impide que usuarios legítimos con conexiones lentas puedan iniciar sesión.

80. ¿Qué técnica de seguridad de API implica añadir una firma digital a las solicitudes, usando una clave privada del cliente, que el servidor puede verificar con la clave pública correspondiente?

- a. Firma de Solicitudes (Request Signing).
- b. Autenticación Básica (Basic Auth) sobre HTTPS.
- c. Uso de tokens JWT firmados por el servidor.
- d. Autenticación mediante API Key en cabecera.

81. Durante la implementación de una API REST, un desarrollador permite que los parámetros del request se concatenen directamente en una

consulta SQL. ¿Cuál es la medida más adecuada para mitigar este riesgo?

- a. Implementar autenticación basada en tokens JWT
- b. Configurar un firewall de aplicaciones (WAF)
- c. Usar consultas parametrizadas o un ORM con binding de variables
- d. Escapar manualmente los caracteres especiales en la cadena de consulta

82. ¿Cuál es el beneficio de seguridad de usar contenedores Docker con el flag '--read-only' para el sistema de archivo raíz?

- a. Permite al contenedor escribir logs en cualquier ubicación.
- b. Facilita la actualización de paquetes dentro del contenedor en ejecución.
- c. Previene modificaciones en el sistema de archivos del contenedor en tiempo de ejecución, dificultando la persistencia de malware o la alteración de archivos críticos.
- d. Mejora el rendimiento de lectura/escritura del contenedor

83. En el contexto de la seguridad en la nube, ¿qué describe el "Modelo de Responsabilidad Compartida (Shared Responsibility Model)?"

- a. Un acuerdo legal que exime al proveedor de nube de cualquier responsabilidad por brechas de seguridad.
- b. Un modelo donde toda la seguridad es gestionada automáticamente por el proveedor de la nube.
- c. Un modelo donde toda la seguridad recae únicamente en el cliente.
- d. Define qué aspectos de la seguridad son responsabilidad del proveedor de la nube (ej. seguridad *de* la nube: infraestructura física, hipervisor) y cuáles son responsabilidad del cliente (ej. seguridad *en* la nube: datos, configuración de SO, red, IAM).

84. En seguridad de IA, ¿qué se entiende por "Privacidad Diferencial" (Differential Privacy)?

- a. Una técnica matemática que permite realizar análisis sobre conjuntos de datos agregados añadiendo *ruido* controlado, de manera que se obtienen resultados útiles sin revelar información sobre individuos específicos en el dataset.
- b. El cifrado de los datos antes de enviarlos al modelo de IA.

- c. Un método para entrenar modelos de IA sin usar ningún dato personal.
- d. Una política que exige eliminar todos los datos personales después de usarlos.

85. En el contexto de AI Security, ¿qué describe un "ataque de evasión" (evasion attack)?

- a. Un atacante roba el modelo de IA completo (extracción de modelo).
- b. Un atacante inyecta datos maliciosos durante la fase de entrenamiento del modelo.
- c. Un atacante deduce información sensible sobre los datos de entrenamiento consultando el modelo.
- d. Un atacante modifica ligeramente la entrada (ej. una imagen) para que el modelo de IA la clasifique incorrectamente durante la inferencia (en producción).